

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

Department of Computer Science & Engineering | Lab Internal I Complete Solution Manual

Course: Software Engineering (23CC501PC / 23CC505PC) • Semester: III Year / I Semester (A.Y. 2026-2027) • All Question Sets (SET-1 through SET-9) Included

SET 1: Hospital Management System (HMS)

Faculty: K. Yasaswini | Branch: CSE - A | Code: 23CC505PC | Repo: <https://github.com/ssvkotamraju/HMS.git>

Part I: Maven Build Execution & Troubleshooting (40 Marks)

1. Clone the HMS Source Code and Import into Eclipse

Terminal Command:

```
git clone https://github.com/ssvkotamraju/HMS.git
cd HMS
```

Eclipse Import Procedure:

1. Open Eclipse IDE → Click **File** → **Import...**
2. Expand **Maven** folder → Select **Existing Maven Projects** → Click **Next**.
3. Click **Browse...** and select the cloned root folder where **pom.xml** is located.
4. Ensure the **pom.xml** checkbox is ticked and click **Finish**.
5. Verify the project icon shows a small **M** overlay and the *Maven Dependencies* library appears under the project tree.

2. "'mvn' is not recognized" Error Diagnosis and Environment Setup

Cause: The Apache Maven **bin** directory has not been added to the system's **PATH** environment variable, or **JAVA_HOME** / **M2_HOME** are not configured.

Configuration Steps (Windows):

1. Open **System Properties** → **Advanced** → **Environment Variables**.
2. Under System Variables, verify/create **JAVA_HOME** pointing to the JDK install directory (e.g., **C:\Program Files\Eclipse Adoptium\jdk-17.0.8.7-hotspot**).
3. Create **MAVEN_HOME** (or **M2_HOME**) pointing to the Maven root folder (e.g., **C:\apache-maven-3.9.6**).
4. Locate the **Path** variable, click **Edit** → **New**, and append **%MAVEN_HOME%\bin**.
5. Restart terminal and verify using:

```
mvn -version
```

3. Unresolved Dependencies in Eclipse & pom.xml Fixes

Diagnosis & Fixes:

- Inspect the **Markers View** or **Problems View** in Eclipse to identify missing dependency coordinates.
- Verify there are no typos in **<groupId>**, **<artifactId>**, or **<version>** inside **pom.xml**.
- Check if dependencies require a specific repository (e.g., Spring milestone or custom Maven repo) and declare **<repositories>** if required.
- Force refresh dependencies: Right-click Project → **Maven** → **Update Project...** (**Alt+F5**) → Tick **Force Update of Snapshots/Releases** → Click **OK**.
- Terminal command: **mvn clean install -U**.

4. Maven Compilation Failure Troubleshooting

Run the build with error stack traces and debug output:

```
mvn clean compile -e -X
```

- **-e** provides execution exception stack traces pinpointing the exact Java file and line number failing compilation.
- **-X** enables full debug logging, displaying active compiler plugin arguments, bootclasspath, and resolved JARs.

- Check compiler output for syntax errors, missing classes, or incompatibility between the Java source level and language features used.

5. Java Version Compatibility Resolution

Configure Java source and target versions directly in `pom.xml` properties or compiler plugin:

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <release>17</release>
      </configuration>
    </plugin>
  </plugins>
</build>
```

In Eclipse: Right-click project → *Properties* → *Java Compiler* → Enable project-specific settings and set Compiler compliance level to 17.

6. Skipping Tests During Packaging & Associated Risks

Command:

```
mvn clean package -DskipTests
```

(Or `mvn clean package -Dmaven.test.skip=true` to avoid compiling test source code as well).

Risks: Skipping tests may deploy undetected syntax/runtime bugs, broken business logic, or corrupted database mapping files directly to production servers.

7. Identifying and Resolving Dependency Conflicts

1. Display dependency hierarchy: `mvn dependency:tree -Dverbose`. Maven marks conflicting omitted versions with `(omitted for conflict with ...)`.
2. Maven resolves conflicts by *Nearest Definition*. To force a specific library version, declare it explicitly in `<dependencyManagement>` or add an `<exclusion>` in `pom.xml`:

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-web</artifactId>
  <version>5.3.30</version>
  <exclusions>
    <exclusion>
      <groupId>commons-logging</groupId>
      <artifactId>commons-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

8. Unexpected Behavior After Dependency Modification

1. Run `mvn dependency:tree` to check if the new dependency brought unwanted transitive dependencies.
2. Purge cached local artifacts: `mvn dependency:purge-local-repository`.
3. Check for class clashes (such as multiple SLF4J bindings or overlapping Servlet API jars).

9. "BUILD SUCCESS" but Application Fails to Open in Browser

1. **Port & Process:** Confirm server is actively listening on port 8080 (`netstat -ano | findstr :8080`).
2. **Context Path:** Verify the application URL. If the WAR file is `HMS.war`, browse to `http://localhost:8080/HMS/` instead of root.
3. **Server Logs:** Inspect Tomcat logs (`catalina.out` or `localhost.log`) for `ServletException` or context initialization failure.

4. **Package Integrity:** Inspect the WAR archive inside `target/` and confirm `WEB-INF/web.xml` and classes exist.

10. Build Plugin Cannot Be Found or Executed

Inspect the following in `pom.xml`:

- Ensure `<groupId>` and `<artifactId>` are valid.
- Ensure the plugin is declared inside `<build><plugins>` and not only inside `<pluginManagement>`.
- Check if the plugin execution goal is tied to a valid build lifecycle phase in `<executions>`.

Part II: Working with Version Control (Git & GitHub) (40 Marks)

1. Initialize Git in an Unversioned Folder:

```
cd HMS-project
git init
git add .
git commit -m "Initial commit for Hospital Management System"
```

2. Git Workflow for Mobile Number & Password Validation:

```
git checkout -b feature/patient-registration-update
# (Modify src/main/webapp/register.jsp)
git status
git add src/main/webapp/register.jsp
git commit -m "Add mobile number field and client-side password validation"
git push -u origin feature/patient-registration-update
```

3. Inspect Unwanted Modification Before Committing:

```
git diff src/main/webapp/register.jsp
```

Output displays removed lines preceded by `-` in red, and added lines preceded by `+` in green.

4. Prepare Untracked and Modified Files for Commit:

```
git add src/main/webapp/register.jsp src/main/webapp/css/register.css
git status
```

5. Undo Commit Breaking Registration Page While Preserving History:

```
git revert HEAD --no-edit
```

Generates a new inversion commit that undoes the modifications while keeping full historical audit trail.

6. Remove Staged Configuration File Without Deleting from Disk:

```
git restore --staged src/main/resources/temp-config.env
# Alternative: git reset HEAD src/main/resources/temp-config.env
```

7. Urgent Fix on Another Branch Mid-Work:

```
git stash push -m "WIP changes on register.jsp"
git checkout hotfix/appointment-booking
# (Fix bug, stage and commit)
git checkout feature/patient-registration-update
git stash pop
```

8. Clean Project History via Rebase:

```
git checkout main
git pull origin main
git checkout feature/patient-registration-update
git rebase main
```

9. Recover Lost Commit:

```
git reflog
# Locate the commit SHA-1 where work was safe (e.g., HEAD@{3} or a1b2c3d)
git checkout -b recovery-branch a1b2c3d
# Or cherry-pick: git cherry-pick a1b2c3d
```

10. Configure Remote Repository and Push Changes:

```
git remote add origin https://github.com/your-username/HMS.git
git branch -M main
git push -u origin main
```

Part III: Run the Project with Docker (20 Marks)

1. Docker Environment Consistency: Docker encapsulates the entire runtime—including the OS layer, specific OpenJDK 17 binaries, Tomcat application server configuration, and environment dependencies—into a portable image. This ensures identical execution regardless of the host developer's operating system or software versions.

2. Dockerfile Instructions for HMS:

```
FROM tomcat:9.0-jdk17-temurin
LABEL maintainer="kmit-admin@hms.org"
WORKDIR /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/HMS.war ./ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

3. Troubleshooting Docker Build Failures:

- Verify that `mvn clean package` was run and `target/HMS.war` exists.
- Check instruction syntax in `Dockerfile`.
- Build with full logs: `docker build --no-cache --progress=plain -t hms:v1 .`

4. Port Mapping Explanation: By default, containers use isolated network namespaces. Using `-p 8080:8080` maps port 8080 on the host machine to port 8080 inside the Tomcat container.

```
docker run -d -p 8080:8080 --name hms-app hms:v1
```

5. Tag and Push Image to Docker Hub:

```
docker tag hms:v1 yourdockerhubusername/hms:v1
docker login
docker push yourdockerhubusername/hms:v1
```

SET 2: E-Ticketing System (OTS)

Faculty: K. Yasaswini | Branch: CSE - A | Code: 23CC505PC | Repo: <https://github.com/ssvkotamraju/E-ticketing.git>

Part I: Maven Build Execution & Troubleshooting (40 Marks)

1. Clone & Import Steps:

```
git clone https://github.com/ssvkotamraju/E-ticketing.git
```

In Eclipse: *File* → *Import* → *Existing Maven Projects*, choose root folder containing `pom.xml`, check the box and click *Finish*.

- 2. Identify & Resolve Maven Dependencies:** Inspect `pom.xml` for missing dependencies or version mismatches. Right-click project → *Maven* → *Update Project (Alt+F5)*.
- 3. Java Compiler Version Mismatch:** Update compiler source/target in `pom.xml` to match installed JDK (e.g., 17):

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>
```

- 4. Compilation Failure Root Cause:** Run `mvn clean compile -e`. Scan the console for `[ERROR] COMPILATION ERROR` to pinpoint missing symbols, syntax errors, or incompatible API interfaces.
- 5. Modify Unexpected Package Type:** Locate `<packaging>` in `pom.xml`. Change from `<packaging>jar</packaging>` to `<packaging>war</packaging>`.
- 6. Why Clean Build Helps & Command:** Stale compiled `.class` files or obsolete resources can cause runtime errors. Running `mvn clean package` deletes the `target/` directory and rebuilds fresh artifacts.
- 7. App Doesn't Open in Browser:** Verify Tomcat is running; ensure host port 8080 is not blocked; check application URL (e.g., `http://localhost:8080/E-ticketing/`); inspect Tomcat `catalina.out`.

8. Skip Tests During Packaging:

```
mvn clean package -DskipTests
```

9. Diagnose Dependency Download Failures: Check coordinate spellings on Maven Central; inspect corporate firewall/proxy in `settings.xml`; check `~/.m2/repository` for lock files (`.lastUpdated`).

10. Run Only Tests:

```
mvn test
```

Part II: Working with Version Control (Git & GitHub) (40 Marks)

1. First Commit on Unversioned Railway Folder:

```
git init
git add .
git commit -m "Initial commit for Railway Ticket Booking system"
```

2. Create Separate Train-Search Feature Branch:

```
git checkout -b feature/train-search
# (Work on feature and verify branch status)
git branch
```

3. Review Modified Ticket-Booking Page & CSS:

```
git status
git diff
```

4. Discard Uncommitted Changes in Train-Search Page:

```
git restore src/main/webapp/train-search.jsp
# (Or: git checkout -- src/main/webapp/train-search.jsp)
```

5. Move Branch Back to Earlier Local Commit:

```
git log --oneline -5
git reset --hard <target_commit_hash>
```

(Use `--soft` to retain modified changes staged in index).

6. Identify & Delete Merged Branches:

```
git branch --merged
git branch -d feature/old-completed-feature
git push origin --delete feature/old-completed-feature
```

7. Merge Independent Feature Branches into Main:

```
git checkout main
git merge feature/train-search
git merge feature/passenger-registration
```

8. Create, Inspect, and Apply Git Patch:

```
# Create patch:
git format-patch -1 HEAD --stdout > feature.patch
# Inspect patch:
git apply --stat feature.patch
git apply --check feature.patch
# Apply patch:
git apply feature.patch
```

9. Resolving Conflicting Patch: Apply using 3-way merge: `git apply -3 feature.patch`. Manually resolve conflict markers in the code editor, stage with `git add`, and commit.

10. Upload Project to GitHub:

```
git remote add origin https://github.com/username/railway-ticketing.git
git branch -M main
git push -u origin main
```

Part III: Run the Project with Docker (20 Marks)

a) **Docker Build Error Diagnosis:** Check the terminal output step by step. Ensure `mvn clean package` produced the target WAR, paths in `COPY` match exactly, and base image exists.

b) **Port Mapping Configuration:** The container listens on internal port 8080. To bind it to the host network interface:

```
-p 8080:8080
```

c) **Container Stops Immediately Diagnosis:**

```
docker ps -a
docker logs <container_id>
```

Ensure the container runs a foreground process (e.g., `CMD ["catalina.sh", "run"]`) rather than a background command.

d) **Solving Configuration Inconsistencies:** Docker creates an immutable runtime containing Tomcat, JDK 17, and application dependencies, eliminating host environmental differences.

e) **Dockerfile for Tomcat WAR Deployment:**

```
FROM tomcat:9.0-jdk17-temurin
WORKDIR /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/E-ticketing.war ./ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

SET 3: Online Recruitment System (ORS)

Faculty: K. Yasaswini | Branch: CSE - A | Code: 23CC505PC | Repo: <https://github.com/ssvkotamraju/ORS.git>

Part I: Maven Build Execution & Troubleshooting (40 Marks)

1. **Clone, Import into Eclipse & POM Verification:**

```
git clone https://github.com/ssvkotamraju/ORS.git
```

Import via *File* → *Import* → *Existing Maven Projects*. Ensure Eclipse displays no build path errors and `pom.xml` has the Maven logo.

2. **Problematic Dependency Identification:** Check the *Markers* tab in Eclipse or run `mvn dependency:analyze`. Update versions in `pom.xml` and execute *Maven* → *Update Project*.

3. **Dependency Conflict Resolution:** Run `mvn dependency:tree -Dverbose`. Use `<exclusions>` to eliminate older incompatible transitive dependencies or declare fixed versions in `<dependencyManagement>`.

4. **Java Version Compatibility:** Set Java compiler source and target properties to `17` in `pom.xml`. In Eclipse, right-click project → *Properties* → *Java Build Path* → *Libraries* → set JRE System Library to JavaSE-17.

5. **Clean Project & Refresh Dependencies:**

```
mvn clean install -U
```

6. **Port Settings & Server Diagnosis:** Ensure Tomcat runs on an open port. If port 8080 is blocked, configure `<port>8082</port>` in the `tomcat7-maven-plugin` configuration. Access at `http://localhost:8082/ORS`.

7. **Identify Port 8080 Conflict & Reconfigure:**

```
# Windows:
netstat -ano | findstr :8080
taskkill /PID <PID> /F
# Linux/macOS:
lsof -i :8080
kill -9 <PID>
```

8. **Fix Compiler Configuration for Installed JDK:** Verify JDK via `javac -version`. Set compiler plugin:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.11.0</version>
  <configuration>
    <release>17</release>
  </configuration>
</plugin>
```

9. **Determine Cause of Dependency Download Failure:** Check spelling of groupId/artifactId in Maven Central; verify internet connection and corporate proxy in `~/.m2/settings.xml`; inspect local repository for corrupted `.lastUpdated` files.
10. **Checks for JDK, Maven, and Dependencies:**

```
mvn -v
mvn dependency:tree
```

Part II: Working with Version Control (Git & GitHub) (40 Marks)

1. **Clone Repository, Create Working Branch & Check Remote:**

```
git clone https://github.com/ssvkotamraju/ORS.git
cd ORS
git checkout -b feature/apply-job
git remote -v
```

2. **Retrieve Remote Changes & Check If Behind:**

```
git fetch origin
git status
git log HEAD..origin/main --oneline
```

3. **Update Branch and Rebase:**

```
git fetch origin
git rebase origin/main
```

4. **Conflict Resolution During Rebase:** Open conflicting `apply.jsp` in editor. Resolve between `<<<<<<` and `>>>>>>`. Then:

```
git add src/main/webapp/apply.jsp
git rebase --continue
```

5. **Systematically Resolve All Conflicts & Confirm:** Repeat `git add` and `git rebase --continue` for each conflicting commit until Git reports "Applying: ... successfully rebased".

6. **Restore Missing Changes:**

```
git reflog
git cherry-pick <commit_hash>
```

7. **Cancel Rebase Safely:**

```
git rebase --abort
```

8. **Configure SSH Key & Authenticate GitHub:**

```
ssh-keygen -t ed25519 -C "student@kmit.in"
cat ~/.ssh/id_ed25519.pub
# Paste into GitHub → Settings → SSH Keys
ssh -T git@github.com
git remote set-url origin git@github.com:ssvkotamraju/ORS.git
```

9. **Verify apply.jsp on Remote GitHub:** Use GitHub web UI or compare remote commit diff using `git diff origin/main`.
10. **Appropriate Push After Rebase & Precautions:**

```
git push --force-with-lease origin feature/apply-job
```

Precaution: Never force-push on shared team branches (like `main`), as it overwrites collaborator commits.

Part III: Run the Project with Docker (20 Marks)

a) **Required Info & Dockerfile Instructions:** Determine base image (`tomcat:9-jdk17`), application archive path (`target/ORS.war`), and server port (8080).

```
FROM tomcat:9.0-jdk17-temurin
WORKDIR /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/ORS.war ./ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

b) **Troubleshooting Docker Context:** Ensure WAR is generated prior to docker build; check path in `COPY`; review `.dockerignore`.

c) **Port Mapping Command:**

```
docker run -d -p 8080:8080 --name ors-container ORS
```

d) **Image Optimization:** Use multi-stage builds and lightweight JRE base images (e.g., `alpine` or `slim`) to minimize attack surface and size.

e) **Tag, Push, and Verify Docker Hub:**

```
docker tag ORS:latest myusername/ors:latest
docker login
docker push myusername/ors:latest
```

SET 4: Online Shopping Application

Faculty: M. Nikitha | Branch: CSE - B | Code: 23CC501PC | Repo: <https://github.com/nikithamoturi/OnlineShoppingApp.git>

Part I: Maven Application Development (40 Marks)

Q1. Maven Project Setup and Build (20 Marks)

a) **Clone & Purpose of pom.xml and JSP:**

```
git clone https://github.com/nikithamoturi/OnlineShoppingApp.git
```

`pom.xml` is the declarative project model defining dependencies, build plugins, and compilation settings. The JSP pages (e.g. `products.jsp`, `login.jsp`) render the presentation layer.

b) **Identify Two Dependencies/Plugins & Purpose:**

- `javax.servlet-api`: Provides the interfaces and classes required to handle HTTP requests and responses.
- `maven-compiler-plugin`: Compiles Java source files into bytecode conforming to specified JDK compliance levels.

c) **Execute mvn clean package & Resolve Build Errors:**

```
mvn clean package
```

If `web.xml attribute is required` error occurs, configure `maven-war-plugin`:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-war-plugin</artifactId>
  <version>3.3.2</version>
  <configuration>
    <failOnMissingWebXml>false</failOnMissingWebXml>
  </configuration>
</plugin>
```

d) **Verify Output & Access Pages:** Verify `target/OnlineShoppingApp.war` is created. Deploy to Tomcat and access: `http://localhost:8080/OnlineShoppingApp/index.jsp`, `login.jsp`, `register.jsp`, `products.jsp`.

Q2. Maven Dependency and Build Troubleshooting (20 Marks)

a) **Dependency Tree:** `mvn dependency:tree`

b) Dependency Resolution Rule: Maven applies *Nearest-Definition*. If conflict depths match, the first declared dependency in the POM wins.

c) Local Repository Location: Default location is `~/.m2/repository` (Windows: `C:\Users\<username>\.m2\repository`). Check for complete JARs and valid POM files.

d) Build and Debug Commands:

```
mvn clean compile
mvn clean package -X -e
```

e) Isolating Issues:

- If Maven build succeeds, compilation and packaging are error-free.
- If Tomcat logs show `ClassNotFoundException` or `NullPointerException`, check server runtime dependencies or `web.xml` mapping.
- If page displays raw scriptlets or 500 error, inspect JSP syntax and taglib directives.

Part II: Git & GitHub Integration (40 Marks)

Q3. Git Repository & JSP Modification

```
# a) Clone & status
git clone https://github.com/nikithamoturi/OnlineShoppingApp.git
cd OnlineShoppingApp
git status
git remote -v

# b) Branching
git checkout -b feature/product-update
git branch

# c) Modify products.jsp (Category & Quantity)
git diff src/main/webapp/products.jsp

# d) Commit
git add src/main/webapp/products.jsp
git commit -m "Add Product Category and Quantity fields in products.jsp"
git log -1
```

Q4. Git Synchronization & Merge

```
# a) Diff inspection
git diff

# b) Update local repo without losing commits
git fetch origin
git rebase origin/main # Or: git merge origin/main

# c) Compare feature branch with main
git diff main...feature/product-update

# d) Merge into main and push
git checkout main
git pull origin main
git merge feature/product-update
git push origin main
```

Part III: Containerization and Container Management (20 Marks)

Q5. Dockerfile & Containerization

```
FROM tomcat:9.0-jdk17-temurin
WORKDIR /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/*.war ./ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

Build image:

```
docker build -t OnlineShopping:v1 .
docker images
```

Q6. Run, Verify and Push Docker Image

```
# a & b) Run container and verify
docker run -d -p 8080:8080 --name onlineshopping-app OnlineShopping:v1
docker ps
# Open browser at http://localhost:8080

# c & d) Push to Docker Hub
docker tag OnlineShopping:v1 myusername/onlineshopping:v1
docker login
docker push myusername/onlineshopping:v1
```

SET 5: Blood Bank Management System

Faculty: S. Srijja | Branch: CSE | Code: 23CC505PC | Repo: <https://github.com/sarasrija/Blood-Bank-Management-system.git>

Part I: Git & GitHub Collaboration (20 Scenarios × 2M = 40 Marks)

#	Scenario Requirement	Exact Git Terminal Command(s)
1	Set up codebase on local machine	<code>git clone https://github.com/sarasrija/Blood-Bank-Management-system.git</code>
2	Verify configured remote server connections	<code>git remote -v</code>
3	Inspect staged, unstaged, and untracked changes	<code>git status</code>
4	Move directly to <code>feature/donor-registration</code>	<code>git checkout -b feature/donor-registration</code>
5	Determine all branches & confirm active branch	<code>git branch -a</code>
6	Commit <code>DonorServlet.java</code> with specific message	<code>git add src/main/java/com/bloodbank/servlet/DonorServlet.java</code> <code>git commit -m "Add donor registration servlet"</code>
7	Merge <code>web.xml</code> into last snapshot without altering note	<code>git add src/main/webapp/WEB-INF/web.xml</code> <code>git commit --amend --no-edit</code>
8	Retrieve remote updates without modifying working files	<code>git fetch origin</code>
9	Incorporate updates from remote main into active branch	<code>git merge origin/main</code>
10	Rebase feature branch history onto updated main	<code>git rebase main</code>
11	Cancel rebase entirely and return to previous state	<code>git rebase --abort</code>
12	Undo faulty commit in <code>DonorTest.java</code> preserving log	<code>git revert <commit_hash> --no-edit</code>
13	Undo last commit keeping changes staged in index	<code>git reset --soft HEAD~1</code>
14	Remove <code>db-config.env</code> from Git keeping local file	<code>git rm --cached src/main/resources/db-config.env</code>
15	Safely stash uncommitted <code>inventory.jsp</code> changes	<code>git stash push -m "WIP inventory.jsp modifications"</code>
16	View all stashes and restore saved modifications	<code>git stash list</code> <code>git stash pop</code>
17	Line-by-line diff on <code>index.jsp</code> between feature and main	<code>git diff main..feature/donor-registration -- src/main/webapp/index.jsp</code>
18	Compact, single-line visual timeline mapping branch merges	<code>git log --oneline --graph --decorate --all</code>
19	Return to main and merge completed feature branch	<code>git checkout main</code> <code>git merge feature/donor-registration</code>
20	Upload updated main to GitHub and verify identical commit point	<code>git push origin main</code> <code>git status</code>

Part II: Maven Java Application Development (40 Marks)

Section A: Project Debugging & Inspection [12 Marks]

Q1. POM Validation & Plugin Debugging [6M]

- **XML Schema Violation (3M):** Missing mandatory child elements (such as `<artifactId>`) or misnested tags in `<dependency>`.
Correct configuration:

```
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>4.0.1</version>
  <scope>provided</scope>
</dependency>
```

- **Tomcat Plugin Failure (3M):** `mvn tomcat7:run` fails if the plugin is undeclared or missing its groupId/version inside `<build><plugins>`. Correct block:

```
<plugin>
  <groupId>org.apache.tomcat.maven</groupId>
  <artifactId>tomcat7-maven-plugin</artifactId>
  <version>2.2</version>
  <configuration>
    <port>8080</port>
    <path>/bloodbank</path>
  </configuration>
</plugin>
```

Q2. Dependency Resolution & Lifecycle Failures [6M]

- **Database Driver Resolution (3M):**

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.2.0</version>
</dependency>
```

- **Missing Test Scope/Artifact (3M):** Test-compile fails when JUnit is missing:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.1</version>
  <scope>test</scope>
</dependency>
```

Section B: Deployment, Configuration & Architecture [10 Marks]

Q3. Build Output Analysis [5M]

- **Artifact Name Rule (3M):** Governed by `<finalName>` inside `<build>`. If omitted, default rule is `${artifactId}-${version}.${packaging}` (e.g., `Blood-Bank-Management-system-0.0.1-SNAPSHOT.war`).
- **Context Path URL (2M):** Determined by plugin configuration (e.g., `http://localhost:8080/bloodbank`).

Q4. Maven Core Concepts [5M]

- **<pluginManagement> vs <plugins> (3M):** `<pluginManagement>` provides central configuration and version locking for plugins without binding them to the build lifecycle; `<plugins>` directly activates execution during build phases.
- **SNAPSHOT Versioning (2M):** `-SNAPSHOT` denotes an active development version that Maven periodically checks and updates from remote repositories, unlike immutable release versions.

Section C: Command Line Operations & Custom Libraries [10 Marks]

- **Q5. Debugging Flags (5M):** JDK inspection command: `mvn -v` (2M). Clean, package with full debug: `mvn clean package -X` (3M).

- **Q6. Local Repository Operations (5M):** Install unindexed JAR:

```
mvn install:install-file -Dfile=blood-analytics.jar -DgroupId=com.vendor.analytics -
DartifactId=blood-analytics -Dversion=1.0.0 -Dpackaging=jar
```

Display dependency tree: `mvn dependency:tree` (2M).

Section D: Testing Protocols & Dependency Management [8 Marks]

- **Q7. Test Execution Controls (4M):** Compiled tests: `target/test-classes`; Reports: `target/surefire-reports` (2M).
Run single test: `mvn test -Dtest=BloodInventoryTest` (1M).
Continue packaging on test failure: `-Dmaven.test.failure.ignore=true` (1M).
- **Q8. Dependency Scope Management (4M):** Analyze dependencies: `mvn dependency:analyze` (1M).
Servlet API `provided` scope: Tomcat includes servlet classes in runtime `lib/`. Bundling them creates ClassCastException and classloader collisions (3M).

Part III: Dockerization and Container Management (20 Marks)

1. Clone & confirm root workspace: `git clone https://github.com/sarasrija/Blood-Bank-Management-system.git && cd Blood-Bank-Management-system && ls -la` [2M]
2. Create Dockerfile: [3M]

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/*.war /app/Blood-managent.war
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/Blood-managent.war"]
```
3. Build image: `docker build -t Bloodbankapp-image .` [2M]
4. Run container: `docker run -d -p 8080:8080 --name Bloodbank-app-container Bloodbankapp-image` [2M]
5. Display running containers: `docker ps` [1M]
6. Display all containers: `docker ps -a` [1M]
7. Interactive shell: `docker exec -it Bloodbank-app-container /bin/bash` [2M]
8. Stop and restart: `docker stop Bloodbank-app-container && docker start Bloodbank-app-container` [2M]
9. Commit container snapshot: `docker commit 0e993d2009a1 your_dockerhub_username/Bloodbankapp:v1` [2M]
10. Authenticate Docker Hub: `docker login` [1M]
11. Upload image: `docker push your_dockerhub_username/Bloodbankapp:v1` [1M]
12. Terminate session & logout: `docker logout` [1M]

SET 6: Sports Management System

Faculty: S. Srija | Branch: CSE | Code: 23CC505PC | Repo: <https://github.com/sarasrija/SportManagementSystem.git>

Part I: Git & GitHub Collaboration (20 Scenarios = 40 Marks)

#	Scenario Requirement	Exact Git Terminal Command(s)
1	Set up codebase locally	<code>git clone https://github.com/sarasrija/SportManagementSystem.git</code>
2	Verify remote connections	<code>git remote -v</code>
3	Inspect staged, unstaged, untracked changes	<code>git status</code>
4	Create and move to <code>feature/player-registration</code>	<code>git checkout -b feature/player-registration</code>
5	Determine branches & confirm active branch	<code>git branch -a</code>
6	Commit <code>RegistrationServlet.java</code>	<code>git add src/main/java/com/sports/servlet/RegistrationServlet.java</code> <code>git commit -m "Add player registration servlet"</code>
7	Merge <code>web.xml</code> into last snapshot without changing message	<code>git add src/main/webapp/WEB-INF/web.xml</code> <code>git commit --amend --no-edit</code>
8	Fetch remote changes without modifying local files	<code>git fetch origin</code>
9	Merge remote main into active branch	<code>git merge origin/main</code>
10	Rebase feature branch history onto updated main	<code>git rebase main</code>
11	Cancel rebase on conflict	<code>git rebase --abort</code>
12	Safely undo faulty commit in <code>ScoreTest.java</code>	<code>git revert <commit_hash> --no-edit</code>
13	Undo last commit keeping changes staged in index	<code>git reset --soft HEAD~1</code>
14	Remove <code>db-config.env</code> from tracking keeping local file	<code>git rm --cached src/main/resources/db-config.env</code>
15	Stash uncommitted changes in <code>schedule.jsp</code>	<code>git stash push -m "WIP schedule.jsp modifications"</code>
16	List and restore stashed work	<code>git stash list</code> <code>git stash pop</code>
17	Line-by-line diff on <code>index.jsp</code> between feature and main	<code>git diff main..feature/player-registration -- src/main/webapp/index.jsp</code>
18	Compact, single-line visual timeline	<code>git log --oneline --graph --decorate --all</code>
19	Return to main and bring feature changes into main	<code>git checkout main</code> <code>git merge feature/player-registration</code>
20	Push main to GitHub and verify identical commit point	<code>git push origin main</code> <code>git status</code>

Part II: Maven Java Application Development (40 Marks)

- **Q1. POM Validation:** Correct invalid XML schema closing tags; add `tomcat7-maven-plugin` under `<build><plugins>`.
- **Q2. Dependencies:** Add MySQL connector dependency; add JUnit test framework dependency with `<scope>test</scope>`.
- **Q3. Build Output:** File is named `sports-management.war` (via `<finalName>`) or `SportManagementSystem-0.0.1-SNAPSHOT.war`. Access context path: `http://localhost:8080/sports`.
- **Q4. Concepts:** `<pluginManagement>` centralizes declarations; `<plugins>` executes them. SNAPSHOT allows mutable development iterations.
- **Q5. Debugging:** Check JDK: `mvn -v`. Clean package with debug: `mvn clean package -X`.
- **Q6. Local Repo:** Install vendor JAR: `mvn install:install-file -Dfile=sports-analytics.jar -DgroupId=com.vendor -DartifactId=sports-analytics -Dversion=1.0.0 -Dpackaging=jar`. Dependency tree: `mvn dependency:tree`.
- **Q7. Testing:** Test classes: `target/test-classes`; Reports: `target/surefire-reports`. Run single test: `mvn test -Dtest=TournamentBracketTest`. Flag: `-Dmaven.test.failure.ignore=true`.
- **Q8. Scope:** Command: `mvn dependency:analyze`. Servlet API is `provided` because container provides it at runtime.

Part III: Dockerization and Container Management (20 Marks)

1. Clone & confirm files: `git clone https://github.com/sarasrija/SportManagementSystem.git && cd SportManagementSystem && ls -la` [2M]
2. Dockerfile: [3M]

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/sports-management.jar /app/sports-management.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "sports-management.jar"]
```

3. Build image: `docker build -t sportsapp-image .` [2M]
4. Launch container: `docker run -d -p 8080:8080 --name sports-app-container sportsapp-image` [2M]
5. Running containers: `docker ps` [1M]
6. All containers: `docker ps -a` [1M]
7. Shell session: `docker exec -it sports-app-container /bin/bash` [2M]
8. Stop & restart: `docker stop sports-app-container && docker start sports-app-container` [2M]
9. Commit snapshot: `docker commit 0e993d2009a1 your_dockerhub_username/sportsapp:v1` [2M]
10. Authenticate: `docker login` [1M]
11. Push image: `docker push your_dockerhub_username/sportsapp:v1` [1M]
12. Logout: `docker logout` [1M]

SET 7: Apartment Management System

Faculty: S. Srija | Branch: CSE | Code: 23CC505PC | Repo: <https://github.com/sarasrija/ApartmentManagementSystem.git>

Part I: Git & GitHub Collaboration (20 Scenarios = 40 Marks)

#	Scenario Requirement	Exact Git Terminal Command(s)
1	Set up codebase locally	<code>git clone https://github.com/sarasrija/ApartmentManagementSystem.git</code>
2	Verify remote server connections	<code>git remote -v</code>
3	Inspect staged/unstaged/untracked changes	<code>git status</code>
4	Switch directly to <code>feature/tenant-registration</code>	<code>git checkout -b feature/tenant-registration</code>
5	Confirm branches & active branch	<code>git branch -a</code>
6	Commit <code>TenantServlet.java</code>	<code>git add src/main/java/com/apartment/servlet/TenantServlet.java</code> <code>git commit -m "Add tenant registration servlet"</code>
7	Merge <code>web.xml</code> into last snapshot without changing message	<code>git add src/main/webapp/WEB-INF/web.xml</code> <code>git commit --amend --no-edit</code>
8	Fetch remote updates without modifying working files	<code>git fetch origin</code>
9	Incorporate updates from remote main	<code>git merge origin/main</code>
10	Rebase feature branch history onto updated main	<code>git rebase main</code>
11	Cancel rebase entirely upon conflicts	<code>git rebase --abort</code>
12	Revert faulty commit in <code>TenantTest.java</code>	<code>git revert <commit_hash> --no-edit</code>
13	Undo last commit keeping changes staged in index	<code>git reset --soft HEAD~1</code>
14	Remove <code>db-config.env</code> from version control keeping local file	<code>git rm --cached src/main/resources/db-config.env</code>
15	Stash uncommitted changes in <code>complaints.jsp</code>	<code>git stash push -m "WIP complaints.jsp"</code>
16	List stashes and restore saved modifications	<code>git stash list</code> <code>git stash pop</code>
17	Line-by-line diff on <code>index.jsp</code> between feature and main	<code>git diff main..feature/tenant-registration -- src/main/webapp/index.jsp</code>
18	Single-line visual graph timeline	<code>git log --oneline --graph --decorate --all</code>
19	Return to main and merge feature branch	<code>git checkout main</code> <code>git merge feature/tenant-registration</code>
20	Push main to GitHub and verify synchronization	<code>git push origin main</code> <code>git status</code>

Part II: Maven Java Application Development (40 Marks)

- **Q1:** Fix XML element structure in `pom.xml`; add `tomcat7-maven-plugin` under `<build><plugins>`.
- **Q2:** Declare MySQL dependency; declare JUnit dependency with `<scope>test</scope>`.
- **Q3:** Web archive name: `target/apartment-management.war` (governed by `<finalName>`). Context path: `http://localhost:8080/apartment`.
- **Q4:** `<pluginManagement>` provides central configuration; `<plugins>` triggers execution. SNAPSHOT enables mutable build downloads during development.
- **Q5:** Verify JDK: `mvn -v`. Clean package with debug: `mvn clean package -X`.
- **Q6:** Install local library: `mvn install:install-file -Dfile=apartment-analytics.jar -DgroupId=com.vendor -DartifactId=apartment-analytics -Dversion=1.0.0 -Dpackaging=jar`. Dependency tree: `mvn dependency:tree`.
- **Q7:** Test classes: `target/test-classes`; Reports: `target/surefire-reports`. Run single test: `mvn test -Dtest=MaintenanceBillingTest`. Flag: `-Dmaven.test.failure.ignore=true`.
- **Q8:** Analysis: `mvn dependency:analyze`. Provided scope ensures container-provided classes do not conflict with webapp classes.

Part III: Dockerization and Container Management (20 Marks)

- Clone & confirm root files: `git clone https://github.com/sarasrija/ApartmentManagementSystem.git && cd ApartmentManagementSystem && ls -la` [2M]
- Dockerfile: [3M]

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/apartment-management.jar /app/apartment-management.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "apartment-management.jar"]
```
- Build image: `docker build -t apartmentapp-image .` [2M]
- Launch container: `docker run -d -p 8080:8080 --name apartment-app-container apartmentapp-image` [2M]
- Display running: `docker ps` [1M]
- Display all: `docker ps -a` [1M]
- Shell session: `docker exec -it apartment-app-container /bin/bash` [2M]
- Stop & restart: `docker stop apartment-app-container && docker start apartment-app-container` [2M]
- Commit container snapshot: `docker commit 0e993d2009a1 your_dockerhub_username/apartmentapp:v1` [2M]
- Authenticate: `docker login` [1M]
- Push image: `docker push your_dockerhub_username/apartmentapp:v1` [1M]
- Terminate session & logout: `docker logout` [1M]

SET 8: Vehicle Rental Management System

Faculty: M. Nikitha | Branch: CSE - B | Code: 23CC501PC | Repo: <https://github.com/nikithamoturi/VehicleRentalManagementSystem.git>

Part I: Maven Web Project Development (40 Marks)

Q1. Maven Project Setup and Build (20 Marks)

a) Clone & Identify JSP Pages [4M]:

```
git clone https://github.com/nikithamoturi/VehicleRentalManagementSystem.git
```

Import into Eclipse via *File* → *Import* → *Existing Maven Projects*. The primary web pages under `src/main/webapp/` are: `index.jsp` (home landing page), `login.jsp` (user authentication), `register.jsp` (user signup), and `vehicle.jsp` (vehicle catalog listing).

b) POM Coordinates & Dependencies [4M]:

- `<groupId>`: Organization domain namespace.
- `<artifactId>`: Project identifier (`VehicleRentalManagementSystem`).
- `<version>`: Application version tag (`1.0-SNAPSHOT`).
- `<packaging>`: Web application archive format (`war`).
- Dependencies*: `javax.servlet-api` (provides HTTP servlet classes) and `jstl` (JavaServer Pages Standard Tag Library for rendering vehicle records).

c) Maven Clean & Build Troubleshooting [7M]:

```
mvn clean package
```

If `maven-war-plugin` fails with `Error assembling WAR: webxml attribute is required`, configure the plugin in `pom.xml`:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-war-plugin</artifactId>
  <version>3.3.2</version>
  <configuration>
    <failOnMissingWebXml>>false</failOnMissingWebXml>
  </configuration>
</plugin>
```

d) Verify Build Output & Server Execution [5M]:

Check that `target/VehicleRentalManagementSystem.war` is created. Deploy to Tomcat and access: `http://localhost:8080/VehicleRentalManagementSystem/vehicle.jsp`.

Q2. Maven Dependency and Build Troubleshooting (20 Marks)

a) **Dependency Tree [4M]:** Run `mvn dependency:tree` to display hierarchical dependency relationships.

b) **Dependency Resolution & Conflict Impact [4M]:** Maven applies "*Nearest-Definition First*". Incompatible version resolution can lead to missing class methods (`NoSuchMethodError`) or bytecode conflicts at runtime.

c) **Local Repository Verification [4M]:** Inspect local folder `~/.m2/repository` to confirm required JAR files are fully downloaded without corrupt `.lastUpdated` markers.

d) Build and Troubleshooting Commands [4M]:

```
mvn clean
mvn compile
mvn package -X
```

e) Application Troubleshooting Steps [4M]:

- If Maven build succeeds, compilation and packaging are intact.
- Check Tomcat logs (`logs/catalina.out`) for server port collisions or database connection failures.
- Check browser network tab for 404 (wrong context path) or 500 (JSP syntax / scriptlet runtime exceptions).

Part II: Git & GitHub Integration (40 Marks)

Q3. Git Repository and Vehicle Page Modification (20 Marks)

```
# a) Clone and inspect status [4M]
git clone https://github.com/nikithamoturi/VehicleRentalManagementSystem.git
cd VehicleRentalManagementSystem
git status
git remote -v

# b) Create and checkout feature branch [4M]
git checkout -b feature/vehicle-update
git branch

# c) Modify vehicle.jsp (add Vehicle Type and Availability Status) & inspect diff [6M]
git diff src/main/webapp/vehicle.jsp

# d) Stage, commit and verify [6M]
git add src/main/webapp/vehicle.jsp
git commit -m "Add Vehicle Type and Availability Status to vehicle.jsp"
git log -1 --stat
```

Q4. Git Synchronization and Remote Repository (20 Marks)

```
# a) Make additional modification (Rental Rate) and verify diff [4M]
git diff src/main/webapp/vehicle.jsp

# b) Retrieve latest changes from remote without overwriting local commits [4M]
git fetch origin

# c) Compare feature branch with main [4M]
git diff main...feature/vehicle-update

# d) Merge into main, push to GitHub, and verify [8M]
git checkout main
git pull origin main
git merge feature/vehicle-update
git push origin main
git log --oneline -5
```

Part III: Dockerization and Container Management (20 Marks)

Q5. Dockerfile and Application Containerization (12 Marks)

a, b, c) **Dockerfile Setup [9M]:**

```
FROM tomcat:9.0-jdk17-temurin
WORKDIR /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/*.war ./ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

d) Build Docker Image [3M]:

```
docker build -t vehiclerentalapp:v1 .
docker images | grep vehiclerentalapp
```

Q6. Run, Verify and Push Docker Image (8 Marks)

```
# a & b) Run container and verify browser access [5M]
docker run -d -p 8080:8080 --name vehiclerental-container vehiclerentalapp:v1
docker ps
# Open http://localhost:8080 in browser

# c & d) Tag, push to Docker Hub, and verify [3M]
docker tag vehiclerentalapp:v1 yourusername/vehiclerentalapp:v1
docker login
docker push yourusername/vehiclerentalapp:v1
```

SET 9: Gym Membership Management System

Faculty: M. Nikitha | Branch: CSE - B | Code: 23CC501PC | Repo: <https://github.com/nikithamoturi/GymManagementSystem.git>

Part I: Maven Web Project Development (40 Marks)

Q1. Maven Project Configuration and Build (20 Marks)

a) Clone and Eclipse Import [4M]:

```
git clone https://github.com/nikithamoturi/GymManagementSystem.git
```

Import into Eclipse via *File* → *Import* → *Existing Maven Projects*. Ensure Eclipse resolves project facets and recognizes the Maven Nature.

b) Key POM Elements Explained [5M]:

- `<groupId>`: Unique domain organization package (e.g., `com.kmit.gym`).
- `<artifactId>`: Unique name of the specific project artifact (`GymManagementSystem`).
- `<version>`: Current release/development version iteration (`1.0.0-SNAPSHOT`).
- `<packaging>`: Output packaging format (`war` for servlet-based web projects).
- `<dependencies>`: Container declaring all third-party external libraries required during build and execution.

c) Maven Build Lifecycle & WAR Generation [6M]:

```
mvn clean package
```

Lifecycle phases: `validate` → `compile` → `test` → `package`. The generated file is located in `target/GymManagementSystem.war`.

d) Modify Configuration & Rebuild [5M]:

Update `<finalName>` in `pom.xml`:

```
<build>
  <finalName>GymMembershipApp</finalName>
</build>
```

Re-run `mvn clean package` and verify the generated archive is now `target/GymMembershipApp.war`.

Q2. Maven Dependency and Build Analysis (20 Marks)

a) **Dependency Tree [5M]:** Run `mvn dependency:tree`. Two key dependencies are `javax.servlet-api` (compiling HTTP servlets) and `jstl` (rendering membership plans dynamically).

b) Dependency vs. Plugin [4M]: A *dependency* is a library included in the classpath (e.g., MySQL driver); a *plugin* is a tool that executes tasks during the Maven build lifecycle (e.g., `maven-compiler-plugin`).

c) Maven Operations [5M]:

- Clean build files: `mvn clean`
- Compile project: `mvn compile`
- Package application: `mvn package`
- Skip test execution: `mvn package -DskipTests`

d) Diagnosing Build Failure on Another System [6M]: Check if Maven Central is blocked by firewalls or offline mode. Check for corrupted downloads in `~/.m2/repository`. Resolve by running `mvn clean compile -U`.

Part II: Git & GitHub Integration (40 Marks)

Q3. Git Branching, Modification & Commit Management (20 Marks)

```
# a) Clone and inspect status [4M]
git clone https://github.com/nikithamoturi/GymManagementSystem.git
cd GymManagementSystem
git status
git branch
git remote -v

# b) Create and checkout feature branch [4M]
git checkout -b feature/membership-plans
git branch

# c) Modify membership information (add Membership Type & Status) and check diff [6M]
git diff src/main/webapp/plans.jsp

# d) Stage, commit and inspect log [6M]
git add src/main/webapp/plans.jsp
git commit -m "Add Membership Type and Membership Status to membership plans"
git log -1 --stat
```

Q4. Git Synchronization, Revert and Merge (20 Marks)

```
# a) Working tree vs Staged vs Committed [5M]
git diff          # Working directory vs Staged
git diff --staged # Staged changes vs Commit
git log -1 -p     # Latest commit changes

# b) Update local branch preserving work [5M]
git fetch origin
git rebase origin/main # Or: git stash && git pull && git stash pop

# c) Undo unwanted commit without deleting project files [4M]
git reset --soft HEAD~1 # Preserves all files staged in index
# (Or: git revert <commit_hash> --no-edit if commit was already pushed)

# d) Merge feature branch into main and push [6M]
git checkout main
git pull origin main
git merge feature/membership-plans
git push origin main
git log --oneline --graph -5
```

Part III: Dockerization and Container Management (20 Marks)

Q5. Docker Image Creation and Application Deployment (12 Marks)

a) Dockerfile [4M]:

```
FROM tomcat:9.0-jdk17-temurin
LABEL maintainer="gym-management@kmit.in"
WORKDIR /usr/local/tomcat/webapps/
RUN rm -rf /usr/local/tomcat/webapps/*
COPY target/*.war ./ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

b & c) Build and Run Docker Image [8M]:

```
mvn clean package
docker build -t gymapp:v1 .
docker images
docker run -d -p 8080:8080 --name gym-container gymapp:v1
```

Q6. Docker Container Management and Docker Hub (8 Marks)

```
# a) Access application in browser: http://localhost:8080 [2M]

# b) Inspect running containers and logs [2M]
docker ps
docker logs -f gym-container

# c & d) Tag, authenticate, push to Docker Hub, and verify [4M]
docker tag gymapp:v1 yourusername/gymmanagementsystem:v1
docker login
docker push yourusername/gymmanagementsystem:v1
```